

计算机设计与实践

流水线CPU及SoC设计

任务B：Cache实现、AXI读通道

2026·夏



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- ◆ 了解基于ready-valid握手协议的总线工作原理
- ◆ 熟悉AXI总线读通道的接口信号和读时序
- ◆ 掌握AXI总线读通道的实现方法



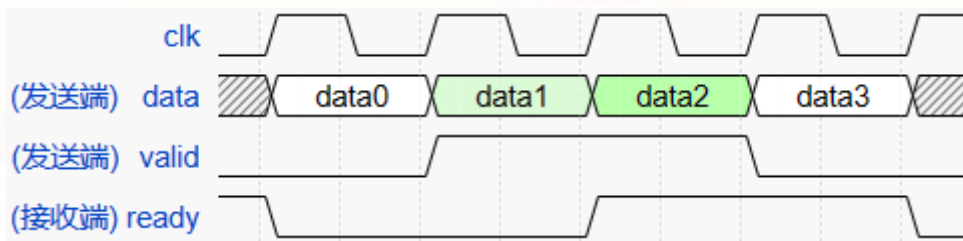
实验内容

- ◆ 把ICache、DCache集成到cpu_top模块
- ◆ 设计AXI总线读通道的状态机
- ◆ 基于“三段式”实现AXI总线读通道
- ◆ 通过功能仿真，完成AXI读通道的初步调试



Ready-Valid握手协议

- ready信号：接收端的就绪状态
- valid信号：发送端发送数据的有效状态

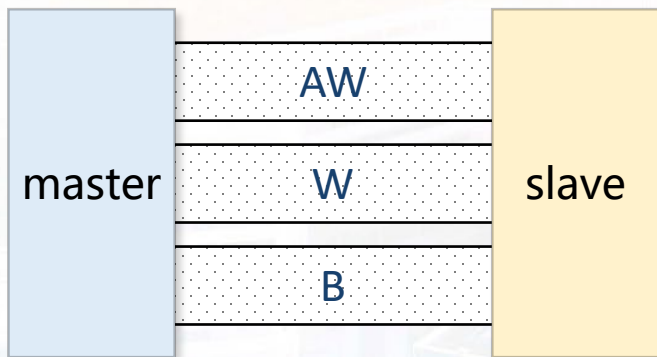


- 对发送端：当且仅当valid有效时，数据才有效 —— data1、data2是有效数据
 - 对接收端：当且仅当ready有效时，才可能接收数据 —— 只有data2被接收
- 当且仅当**valid、ready同时有效**时，数据才能在clk上升沿时成功传输

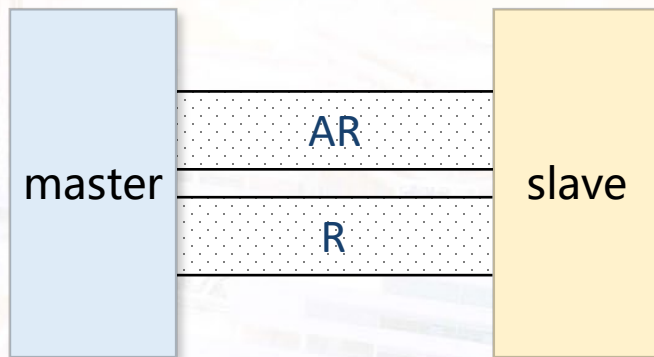
AXI总线协议

❑ 基于ready-valid握手协议的高性能嵌入式总线

- 包含5个独立通道：写地址(**AW**)、写数据(**W**)、写响应(**B**)、读地址(**AR**)、读数据(**R**)



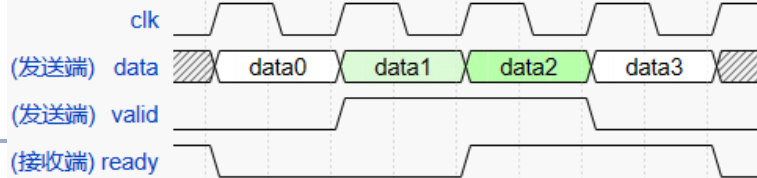
3个通道用于写请求



2个通道用于读请求

- 每个通道都基于ready-valid信号实现握手

AXI读通道



□ AR通道、R通道

- 每个通道都遵循ready-valid握手协议：

		通道	信号	位宽	方向	释义
master 发送 slave 接收	读地址 (AR)		araddr	32	主→从	读地址
			arlen	8	主→从	猝发传输的数据包个数
			arsize	3	主→从	每次传输的数据包大小
			arburst	2	主→从	猝发传输的地址生成方式
			arvalid	1	主→从	读地址的有效信号
			arready	1	从→主	从设备AR通道的就绪信号
slave 发送 master 接收	读数据 (R)		rready	1	主→从	主设备R通道的就绪信号
			rdata	32	从→主	读数据
			rlast	1	从→主	猝发传输最后一个数据包的标志位
			rvalid	1	从→主	读响应的有效信号

AXI读通道

□ AR通道、R通道

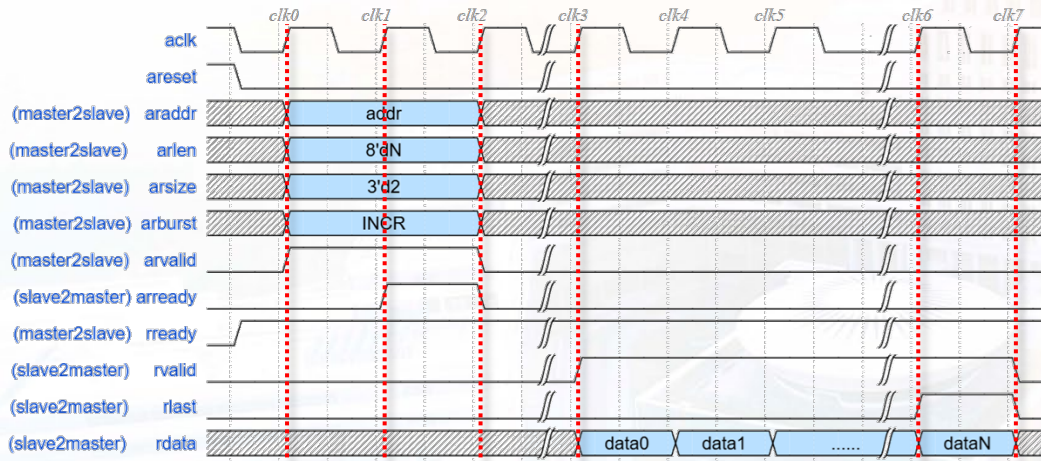
- 猝发传输相关信号：
 - arlen (0~255): 控制猝发传输时连续传输arlen+1个数据包
 - arsize (0~7): 控制猝发传输数据包的大小为 2^{arsize}
 - arburst: 控制猝发传输的地址生成方式

取值	地址生成方式	释义
00	FIXED	固定地址模式，每次访问同一个地址（如FIFO）
01	INCR	递增地址模式，每次访问地址+1（如存储器）
10	WRAP	回环地址模式，地址在一定范围内循环（如缓存器）

- rlast: 指示猝发传输最后一个数据包的标志位信号

AXI读时序

- 主设备: axi_master 从设备: 存储器/外设



【clk0】 master发送读地址, 设定猝发传输参数

【clk1】 slave的AR通道就绪(arready有效), 接收读地址

【clk2】 master撤掉读地址请求; slave执行读操作

【clk3】 读操作完成, slave每个时钟返回1个读数据

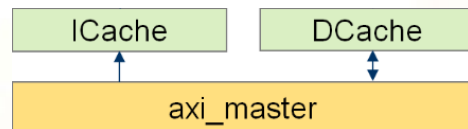
【clk6】 slave返回最后1个数据时拉高rlast信号

【clk7】 slave拉低rvalid和rlast, 传输结束

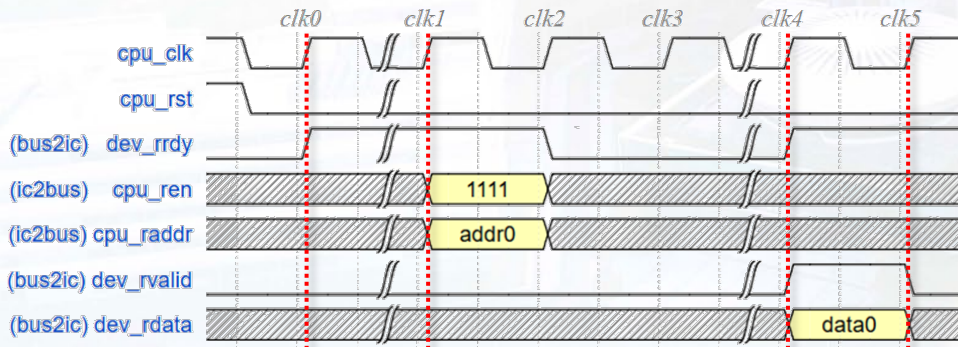
在rlast有效后, 主设备才能发起新请求

Cache的读访存接口

- 主设备：Cache 从设备：axi_master



信号	位宽	方向	释义
dev_rrdy	1	从→主	从设备R通道的就绪信号
cpu_ren	1	主→从	读使能，支持读字、半字和字节
cpu_raddr	32	主→从	读地址
dev_rvalid	1	从→主	读数据的有效信号
dev_rdata	256	从→主	读数据



【clk0】slave就绪，拉高rrdy以接收master访存请求

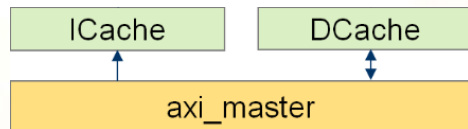
【clk1】master发出请求，slave拉低rrdy，开始处理读请求

【clk4】slave拉高rvalid，把读取的数据块返回给master

【clk5】slave撤掉rvalid，传输结束

Cache的读访存接口

- axi_master同时与ICache、DCache互连

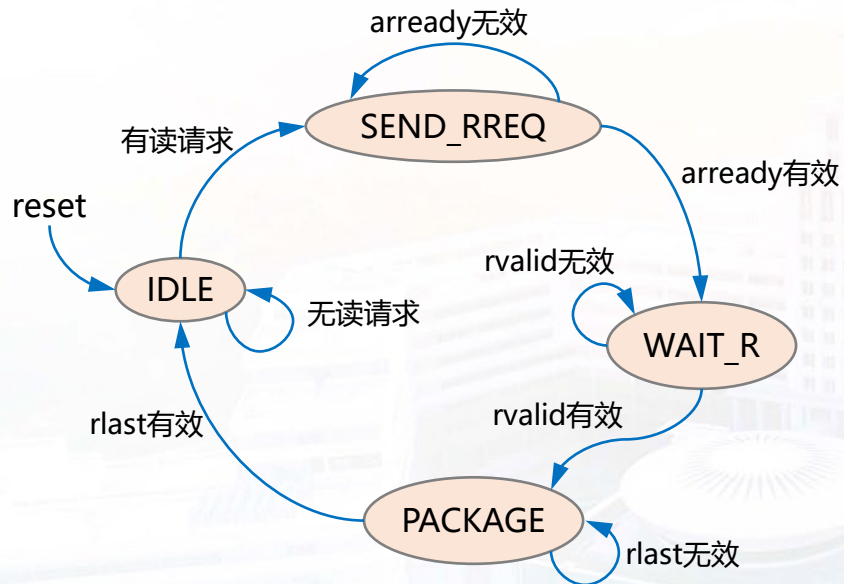


```
1 | // ICache Interface
2 | output reg          ic_dev_rrdy;
3 | input  wire         ic_cpu_rren;
4 | input  wire [ 31:0 ] ic_cpu_raddr;
5 | output reg         ic_dev_rvalid;
6 | output reg [127:0] ic_dev_rdata;
7 | // DCache Read Data Interface
8 | output reg         dc_dev_rrdy;
9 | input  wire         dc_cpu_rren;
10 | input  wire [ 31:0 ] dc_cpu_raddr;
11 | output reg         dc_dev_rvalid;
12 | output reg [127:0] dc_dev_rdata;
```

- 若同时检测到ICache、DCache的访存请求，优先处理DCache的访存请求
- 本实验不考虑同时出现的情形，但实现时注意**DCache的访问请求处理应优先于ICache**

AXI读状态机设计

- 总线控制器处理ICache读请求的状态机参考



状态	描述
IDLE	初始状态，等待接收Cache的读请求
SEND_RREQ	通过AR通道发出读地址请求
WAIT_R	等待从设备通过R通道返回数据
PACKAGE	把从设备返回的数据打包成数据块

- 请自行完成完整读状态机的设计

➤ 注意：DCache读请求优先于ICache读请求、Uncached读

AXI读通道仿真验证

- ◆ 在Vivado中使用默认的lw.coe程序运行功能仿真
- ◆ 把ICache、DCache的读访存接口，以及AXI读通道的相关信号添加到波形，观察波形，分析取指、lw指令的读访存行为能否正确运行
- ◆ 为降低调试难度，建议先在头文件中禁用Cache模块，等lw.coe仿真通过后，再启用Cache模块并再次调试

```
1 // `define ENABLE_ICACHE  
2 // `define ENABLE_DCACHE
```

- ◆ **注意代码规范性，可以避免很多问题！**



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ